Agente de abastecimiento para Pampasur

Orquestación con LangGraph, memoria de corto y largo plazo, y aprobación humana

Evaluación Parcial N°2 · ISY0101 Optativo Ingeniería de Soluciones con IA · **Estudiante:** Felipe Angel · **Fecha:** 21 de octubre de 2026

Repositorio: https://github.com/felipeangel321/pampasur-agente-abastecimiento (código, README, diagramas, pruebas y guion de demostración)

1. Requerimiento organizacional

Pampasur abastece a cinco centros de cultivo que opera para Camanchaca (Maqui, Chaiquén, Peñasmo, Porcelana y Tarao), atendidos por cuidadores en turnos 14x14. En cada relevo, el turno saliente envía por WhatsApp la lista de compra para el turno que entra, y en la práctica esa lista **se copia y pega del turno anterior** sin revisar la bodega. Así se compra lo que todavía queda: detergente que rinde tres turnos o 5 kg de azúcar cuando se consume menos de 1 kg. En el historial simulado del proyecto (73 relevos, abril a octubre de 2026), **el 31% del gasto fueron compras innecesarias** ($5.118.690 de $16.272.550) y 39 de las 68 listas que tenían un relevo anterior eran copias exactas.

La EP1 resolvió la consulta de información con un RAG sobre la política interna, el protocolo de relevo y el catálogo de Lider. Esta EP2 la convierte en un **agente funcional** que automatiza el flujo completo: recibe la lista y el inventario del turno saliente, calcula lo que de verdad hace falta, explica la orden, genera la orden en Excel y un reporte en PDF, y envía los correos solo después de que una persona los aprueba. Las reglas del negocio son: presupuesto de **$180.000 por trabajador por turno para víveres y aseo**, un tope que siempre se respeta; el mantenimiento (pintura, arreglos, luz o agua) es ocasional y se paga como un **extra** que aprueba el Responsable de compras; y en Porcelana y Tarao los víveres viajan en bus con los trabajadores (supuesto: 20 kg por persona).

2. Diseño e implementación del agente (IE1, IE2)

El sistema es un `StateGraph` de LangGraph con ocho nodos y **sin agente manager**: el orden del proceso es conocido, así que vive en el código, y las rutas condicionales dependen de datos del estado (¿llegó lista?, ¿quedan correos?, ¿hay modelo disponible?). Los modelos de lenguaje solo trabajan donde hace falta lenguaje, en dos agentes construidos con `create_agent` de LangChain 1.x: el **Analista**, que explica la orden con la política y la memoria, y el agente de **Correos**, que redacta y envía cada aviso. Las cantidades y los montos los decide el **Comprador** (`src/compra.py`), código Python determinista y probado. La Figura 1 muestra la orquestación completa y la Tabla 1, las herramientas según su función.

Tabla 1. Herramientas de los agentes (siete del Analista y una de Correos).

Tipo	Herramienta	Qué hace	Validación ante entradas inválidas
Consulta	`consultar_centro`	Dotación, presupuesto y traslado de un centro	Centro desconocido: error con la lista de centros
Consulta	`consultar_inventario`	Reporte del turno saliente en una fecha	Fecha sin reporte: lo informa, no inventa
Consulta	`revisar_producto`	Duración, prioridad y necesidad del turno	Mantenimiento: indica que se pide por solicitud
Consulta	`historial_producto`	Cuánto se ha pedido en los últimos relevos	Nombre ambiguo: toma el más parecido del catálogo
Consulta (RAG)	`buscar_politica`	Reglas internas con su cita [archivo > sección]	Sin resultado: lo dice; nunca inventa la regla
Consulta (memoria)	`recordar_relevos`	Relevos anteriores del centro por similitud	Sin memoria: lo dice explícitamente
Razonamiento	`calcular_orden`	Ejecuta el Comprador y devuelve decisiones y alertas	Sin inventario: necesidad completa y alerta
Escritura	`enviar_correo`	Envía por Gmail API con Excel y PDF adjuntos	Solo destinatarios de `contactos.csv` y adjuntos del relevo; requiere aprobación humana

Frameworks y compatibilidad (IE2). LangGraph 1.x aporta el grafo, el checkpointer y el store de memoria (LangChain, s.f.-c, s.f.-d); LangChain 1.x aporta `create_agent` (LangChain, s.f.-a) y su middleware: `HumanInTheLoopMiddleware` para la aprobación (LangChain, s.f.-b), `ModelCallLimitMiddleware` (tope de 6 y 8 llamadas) y `ModelRetryMiddleware` (reintentos ante errores 429 y 400 de Groq). El modelo es `openai/gpt-oss-20b` servido por Groq (Groq, s.f.; OpenAI, 2025), con tool calling compatible con el formato de OpenAI. Los correos salen por la Gmail API (Google, 2026). Los embeddings son locales (paraphrase-multilingual-MiniLM-L12-v2; Reimers y Gurevych, 2020) con índices FAISS (Johnson et al., 2021). La escalabilidad está en los puntos de cambio: el checkpointer y el store de SQLite se reemplazan por Postgres sin tocar el grafo, el proveedor de LLM se cambia en `get_llm` y cada nodo se prueba por separado. El entorno es reproducible con `uv` (Python 3.13) y `uv.lock`.

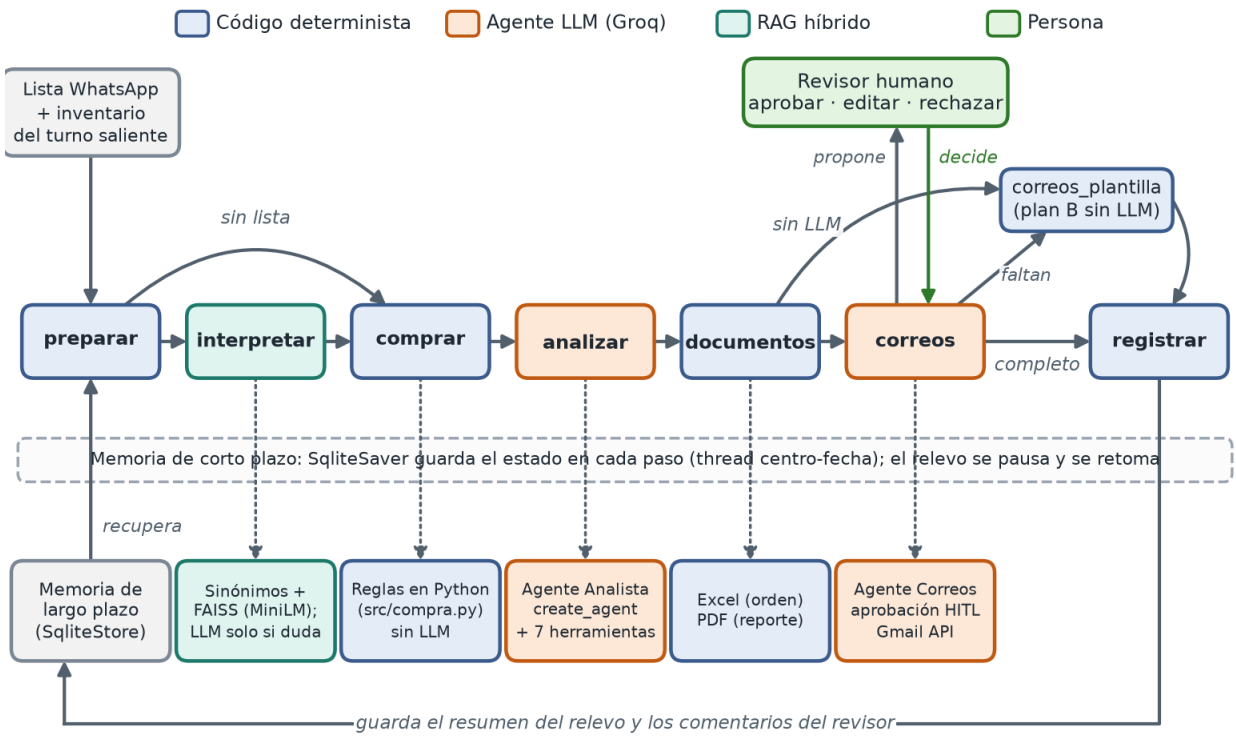


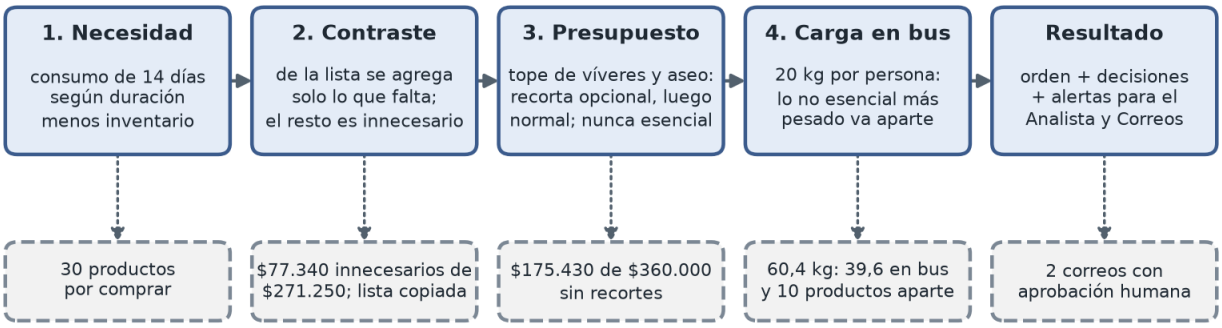
Figura 1. Diagrama de orquestación del agente. Nodos del grafo (fila central), componente que usa cada nodo (fila inferior) y memoria de corto y largo plazo. Elaboración propia a partir de `src/grafo.py`.

3. Memoria y recuperación de contexto (IE3, IE4)

- **Memoria de corto plazo (contenido).** `SqliteSaver` guarda el estado completo del grafo después de cada nodo en `.memoria/checkpoints.sqlite`, con `thread_id = centro-fecha`. Si el revisor elige `[s]alir` durante la aprobación, el relevo se retoma con el mismo comando sin repetir el análisis ni las llamadas al modelo, y un relevo cerrado no se procesa dos veces (`--nuevo` abre otro hilo). El chat del Analista usa un hilo por conversación: a «¿Qué pasó en el último relevo de Tarao?» le sigue «¿Y cuánto detergente quedaba?» y el agente entiende que se sigue hablando de Tarao.
- **Memoria de largo plazo (recuperación semántica).** `SqliteStore` con índice vectorial (sqlite-vec) y los mismos embeddings del RAG. El nodo `registrar` guarda un resumen de cada relevo en `("relevos", centro)` y cada comentario con que el revisor rechazó un correo en `("preferencias", "correos")`. El nodo `preparar` los recupera por similitud al iniciar el relevo siguiente. Ejemplo verificado: el comentario «Pon los kilos por persona en el asunto» se guarda al rechazar un correo y llega como preferencia al agente de Correos en el relevo siguiente (`test_rechazo_con_comentario_el_agente_corrige_y_se_recuerda`).
- **Contexto del dominio.** El RAG híbrido de la EP1 (Lewis et al., 2020) interpreta la lista: primero busca por sinónimos del catálogo, luego por similitud en FAISS y solo pide ayuda al LLM en las líneas dudosas. La herramienta `buscar_politica` devuelve cada regla con su cita, y el Analista debe citarla para afirmarla.

4. Planificación y toma de decisiones (IE5, IE6)

La planificación tiene dos niveles. En el **grafo**, la secuencia de etapas es fija y las rutas condicionales (`ruta_lista`, `ruta_correos`) eligen el camino según el estado: sin lista se salta `interpretar`; si el agente de Correos no envía todo, hay un reintento y después plantillas. En el **Comprador**, la compra sigue las prioridades de la política de abastecimiento (Figura 2). Cada decisión queda escrita en la orden, y el Analista y el agente de Correos la reciben como dato: explican la decisión, pero no la toman.



Ejemplo real del sistema: relevo de Tarao del 2026-10-09

Figura 2. Esquema de planificación del Comprador, con los valores reales del relevo de Tarao del 9 de octubre de 2026. Elaboración propia.

Tabla 2. Decisiones adaptativas demostradas, con su evidencia en ejecuciones y pruebas.

Condición del entorno	Comportamiento del agente	Evidencia
La lista es copia del relevo anterior (90% o más igual)	Alerta «Lista copiada», agregada por código al correo del Responsable	Tarao 41 de 41 y Maqui 39 de 39 productos iguales; test de grafo
Se pide algo que la bodega todavía cubre	No se compra; se informa como compra innecesaria evitada	Tarao: $77.340 evitados de una lista de $271.250
Unidades sueltas («40 rollos de confort»)	Se convierten a paquetes del catálogo	4 paquetes de 12 rollos; test del RAG
Producto fuera del catálogo («2 six de cerveza»)	No se compra y se avisa en el correo	Línea marcada para revisar
Víveres y aseo no caben en el presupuesto	Recorta lo opcional y después lo normal; nunca lo esencial ni pasa el tope	test de presupuesto (Comprador)
Se pide material de mantenimiento	Va como extra, fuera del presupuesto; el correo pide aprobarlo	test del extra de mantenimiento
Centro con bus y compra sobre 20 kg por persona	Lo no esencial más pesado va en envío aparte; correo a los cuidadores con kg por persona	Tarao: 60,4 kg en total, 39,6 kg en el bus
Centro con vehículo	Correo al prevencionista en vez de a los cuidadores	Maqui, Chaiquén y Peñasmo
El revisor rechaza un correo con un comentario	El agente lo corrige y vuelve a pedir aprobación; el comentario queda en memoria	test de rechazo con comentario
Groq no responde o no hay clave	Plan B: resumen calculado y correos de plantilla, con la misma aprobación	test sin LLM

El umbral de lista copiada se calibró con el historial: las 39 listas copiadas coinciden al 100% con la compra anterior y las 29 armadas de nuevo coinciden entre 35% y 91%. Con un umbral de 90% se detectan las 39 copias con 2 falsas alarmas, aceptables porque la alerta solo avisa y no cambia la orden.

5. Justificación de los componentes (IE8)

Tabla 3. Componentes, requerimiento que atienden y justificación.

Componente	Requerimiento del flujo	Por qué se eligió	Alternativa descartada
`StateGraph` sin manager	Etapas conocidas con rutas que dependen de datos	Rutas deterministas, menos tokens y trazabilidad de cada paso	CrewAI jerárquico: 15.000 a 20.000 tokens por mensaje en un proyecto de referencia y delegación no garantizada
Comprador en Python	Montos y cantidades auditables	La misma entrada da siempre la misma orden; 10 pruebas lo cubren	Dejar que el LLM calcule la compra
Agente Analista con 7 herramientas	Explicar la orden con fundamentos	Patrón ReAct (Yao et al., 2023): consulta antes de afirmar y cita la política	Un prompt único con todo el contexto
Aprobación humana (HITL)	El correo tiene efecto externo	Aprobar, editar o rechazar con comentario antes de cada envío	Envío automático
`gpt-oss-20b` en Groq	Tool calling confiable y bajo costo	Sin errores de herramientas; menos de 11.000 tokens por relevo	`gpt-oss-120b`: llamó a una herramienta inexistente (error 400)
Embeddings locales + FAISS	Buscar en documentos internos	Groq no ofrece embeddings; modelo multilingüe sin costo	Embeddings de pago por API
SQLite (checkpointer y store)	Continuidad y memoria semántica en un PC	Persistente y sin servidor; se cambia a Postgres sin tocar el grafo	Memoria en RAM, que se pierde al cerrar
Gmail API (`gmail.send`)	Enviar avisos sin exponer el buzón	Mínimo privilegio: el agente envía pero no puede leer correos	SMTP con contraseña de la cuenta

6. Resultados y evidencia

Tabla 4. Medición de relevos completos con `scripts/medir.py` (Groq, 21 de octubre de 2026).

Relevo	Situación	Llamadas LLM	Tokens entrada / salida	Segundos	Correos
Tarao 2026-10-09	Bus, lista copiada, carga sobre el máximo	5	9.957 / 901	26,7	2
Maqui 2026-10-15	Vehículo, productos de larga duración	5	8.346 / 688	69,4	2
Peñasmo 2026-10-07	Sin lista: solo inventario	5	6.356 / 602	50,3	2

Cada relevo completo usa menos de 11.000 tokens, así que el plan gratuito de 200.000 tokens diarios alcanza para unos 18 relevos al día, cuando una ronda de los cinco centros son 5. Las diferencias de tiempo con igual número de llamadas se deben a la cuota por minuto de Groq, que los reintentos absorben. El repositorio incluye **65 pruebas automatizadas** que pasan (`uv run pytest`): datos, reglas de duración, RAG, Comprador, correo y el grafo completo con un modelo falso, que verifica aprobación, rechazo con comentario, memoria entre relevos y plan B sin gastar tokens. El envío real por Gmail se comprobó con el relevo de Maqui del 15 de octubre: llegaron los dos correos con la orden en Excel y el reporte en PDF. **Limitaciones:** el historial, los inventarios y las listas son simulados con semilla fija; los precios de Lider son referenciales; el permiso de Gmail en modo de prueba vence cada 7 días; y la observabilidad con trazas queda para la EP3.

7. Conclusiones

El agente ataca el problema de raíz: la orden ya no sale de la lista copiada, sino del inventario real y de cuánto dura cada producto. En el relevo de Tarao, de una lista de $271.250 se evitaron $77.340 en compras innecesarias, y el sistema avisó que la lista era una copia del turno anterior. Separar las decisiones (código determinista) de la comunicación (agentes LLM) hizo que el sistema fuera auditable y barato, y la aprobación humana mantiene el control de lo que sale de la empresa. La memoria permite retomar un relevo interrumpido y aprender de los comentarios del revisor. El siguiente paso es usar listas reales anonimizadas y agregar observabilidad en la EP3.

Declaración de uso de IA. Se usó Claude (Anthropic) como asistente de programación y de redacción: apoyó la implementación del código, la documentación y el borrador de este informe. El autor definió el caso, entregó los datos y las reglas de la empresa, tomó las decisiones de alcance (por ejemplo, el grafo sin agente manager y la regla de presupuesto) y verificó el funcionamiento del sistema y de las pruebas.

Referencias

Google. (2026). *Create and send email messages*. Gmail API, Google for Developers. Recuperado el 21 de octubre de 2026, de https://developers.google.com/workspace/gmail/api/guides/sending

Groq. (s.f.). *GPT OSS 20B*. GroqDocs. Recuperado el 21 de octubre de 2026, de https://console.groq.com/docs/model/openai/gpt-oss-20b

Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data, 7*(3), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

LangChain. (s.f.-a). *Agents*. LangChain Docs. Recuperado el 21 de octubre de 2026, de https://docs.langchain.com/oss/python/langchain/agents

LangChain. (s.f.-b). *Human-in-the-loop*. LangChain Docs. Recuperado el 21 de octubre de 2026, de https://docs.langchain.com/oss/python/langchain/human-in-the-loop

LangChain. (s.f.-c). *LangGraph overview*. LangChain Docs. Recuperado el 21 de octubre de 2026, de https://docs.langchain.com/oss/python/langgraph/overview

LangChain. (s.f.-d). *Persistence*. LangChain Docs. Recuperado el 21 de octubre de 2026, de https://docs.langchain.com/oss/python/langgraph/persistence

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

OpenAI. (2025). *gpt-oss-120b & gpt-oss-20b model card* (arXiv:2508.10925). arXiv. https://doi.org/10.48550/arXiv.2508.10925

Reimers, N., & Gurevych, I. (2020). Making monolingual sentence embeddings multilingual using knowledge distillation. En *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 4512–4525). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.365

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. En *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629